

Action 参考

ATP 一共支持 **13 个 action**，这个集合是封闭的，不能扩展。本篇逐个说明用法和易错点。

字段填写速查

不同 action 需要填的字段不同，填错了平台会在保存时报错：

action	locator 必填	input_data	expected	说明
--- --- --- --- ---				
`OPEN_URL`		(URL)		打开页面
`CLICK`				点击元素
`INPUT`				输入文本
`SELECT`				下拉选择
`ASSERT_TEXT`				断言文本内容
`ASSERT_VISIBLE`				断言元素可见
`ASSERT_NOT_EXIST`				断言元素不存在
`WAIT_FOR`				显式等待元素
`SCROLL_TO`				滚动到元素
`SWITCH_FRAME`				切换 iframe
`SWITCH_WINDOW`		(索引/标题)		切换窗口
`UPLOAD`		(文件路径)		文件上传
`SLEEP`		(秒)		规范禁止使用

导航类

OPEN_URL

打开一个页面。`input\_data` 填完整 URL，不需要定位器。

一条案例通常以 `OPEN\_URL` 开始。同一条案例里可以多次使用，用于跨页面的流程测试。

URL 里如果要带动态参数（比如订单号），用数据驱动的变量占位符，不要硬编码。见《数据驱动》。

SWITCH_WINDOW

切换浏览器窗口 / 标签页。`input\_data` 可以填两种值：

- **数字索引** — `0` 是初始窗口，`1` 是第一个新开的窗口，依此类推
- **窗口标题** — 填标题文本，平台会做精确匹配

优先用标题。索引会随着被测系统改动而错位，标题相对稳定，而且看案例的人一眼能知道切到哪去了

。

点击一个 `target="\_blank"` 的链接后，**新窗口不会自动获得焦点**，必须显式 `SWITCH\_WINDOW`。这是新人最常见的困惑之一：明明点开了新页面，后续步骤却报元素找不到——因为操作还在旧窗口上。

SWITCH_FRAME

切换到 iframe 内部。定位器指向 iframe 元素本身。

进入 iframe 后，所有定位都相对于该 iframe，**外层页面的元素将不可见**。要回到主文档，用 `SWITCH\_FRAME` 并把 `locator\_value` 填 `\_\_default\_\_`（这是平台约定的特殊值）。

嵌套 iframe 需要逐层切入，不能一步到位。

交互类

CLICK

点击元素。这是使用频率最高的 action，也是等待策略最容易写错的地方。

`CLICK` 的 `wait\_strategy` **必须**是 `CLICKABLE`，不能是 `VISIBLE` 或 `PRESENCE`。原因是元素「可见」不等于「可点」——它可能被遮挡、被禁用、或者绑定的事件还没注册上。用 `VISIBLE` 会导致偶发的点击无效，而且这类失败非常难复现。

这条规则由平台在保存时自动填充，不需要手工设置。详见《待機戰略規約》。

INPUT

向输入框输入文本。

平台在输入前会**自动清空原有内容**，不需要额外的清空步骤。如果确实需要追加而非覆盖，在 `input\_data` 前加 `+` 前缀，例如 `+追加的文本`。

对于富文本编辑器等非标准输入框，`INPUT` 可能无效，这时改用 `CLICK` 聚焦后再 `INPUT` 通常能解决。

SELECT

操作下拉框。`input\_data` 填**选项的可见文本**，不是 value 属性值。

只支持原生 `` 元素。现在很多前端框架用 `

` 模拟下拉框，那种情况 `SELECT` 用不了，得拆成两步：`CLICK` 展开 + `CLICK` 选中目标项。

UPLOAD

文件上传。定位器指向 `<input type="file">` 元素，`input_data` 填服务端的文件路径。

定位器必须指向真正的 `input` 元素，而不是页面上那个好看的「选择文件」按钮。很多站点会把 `input` 隐藏起来再用按钮触发它，这种情况下定位器要指向**隐藏的那个 input**，并且 `wait_strategy` 用 `PRESENCE` 而不是 `VISIBLE` —— 因为它本来就不可见。

可上传的文件需要事先放在共享存储的 `/testdata/` 目录下，本地路径无效（执行器跑在独立的机器上，读不到你本地的文件）。

SCROLL_TO

滚动到元素位置。

多数情况下不需要显式滚动 —— `CLICK` 会自动把元素滚进视口。需要 `SCROLL_TO` 的典型场景是**触发懒加载**：列表页要滚到底部才会加载下一页。

断言类

ASSERT_TEXT

断言元素的文本内容。`expected` 填期望文本。

默认是**包含匹配**而非完全相等。要精确匹配，在 `expected` 两端加 `^`` 和 ``$`，例如 `^`提交成功`$`。

比较时会自动去掉首尾空白并把连续空白折叠成一个空格，所以不用担心 HTML 里的换行和缩进。

ASSERT_VISIBLE

断言元素存在且可见。不需要 `expected`。

「可见」的判定是：元素在 DOM 中、CSS 没有 `display:none` / `visibility:hidden`、且宽高都大于 0。

注意这不检查元素是否在当前视口内。页面下方需要滚动才能看到的元素，`ASSERT_VISIBLE` 也算通过。

ASSERT_NOT_EXIST

断言元素**不在 DOM 中**。注意它和「不可见」是两回事 —— 一个 `display:none` 的元素是不可见的，但它存在于 DOM 中，`ASSERT_NOT_EXIST` 会失败。

用于验证「删除后列表里没有这一项」「无权限时按钮不渲染」这类场景。

这个断言有个陷阱：如果页面还没加载完，元素当然不存在，断言会「通过」，但这是个假阳性。所以 `ASSERT_NOT_EXIST` 之前应该先有一个能证明页面已就绪的步骤（比如断言某个必然出现的元素可

见)。

等待类

WAIT_FOR

显式等待某个元素达到指定状态。等待条件由 `wait\_strategy` 决定。

多数情况下**不需要单独写 `WAIT\_FOR`** —— 每个 action

自己就带等待。需要它的场景是：等待一个不参与后续操作、但能标志页面就绪的元素，比如加载动画消失。

SLEEP

固定睡眠指定秒数。

** 公司规范明确禁止使用。 ** 平台保留这个 action 只是为了兼容历史案例，新案例一律不允许。

理由是固定等待必然二选一：要么等太久（拖慢整个回归），要么等不够（偶发失败）。而且它掩盖问题 —— 一个需要 `SLEEP 3` 才能通过的步骤，真正的问题是等待策略写错了。

正确做法是用 `wait\_strategy` 显式等待具体条件。详细论证见《待機戰略規約》。

没有的能力

以下是 ATP **不支持**的，遇到需求不要找变通写法，直接反馈到平台组排期：

- **接口 / API 测试** — ATP 只做 Web UI，不发 HTTP 请求，没有断言 JSON 响应的能力
- **移动端 / App 自动化** — 不支持，ATP 只驱动桌面浏览器
- **数据库断言** — 不能直接查库校验数据
- **自定义脚本注入** — 不能执行任意 JavaScript
- **图像比对** — 不做视觉回归

Action 枚举是封闭的，上面表格里的 13 个就是全部。**如果你在别处看到一个不在这张表里的 action 名字，那它不存在** —— 可能是别的平台的写法，或者是记混了。